Knowledge Graph-based Fake News Detection and Analysis System

By Group 3

Isha Sudarshini

Aswini P

Kungumapriya

Varshitha B U

(Infosys Springboard Internship Batch I)

ABSTRACT

In the age of digital communication, misinformation and fake news have become major societal challenges, influencing opinions, shaping public perception, and undermining trust in reliable information sources. To address this issue, this project introduces a **Knowledge Graph-based Fake News Detection and Analysis System** that integrates **Natural Language Processing (NLP)**, **graph theory**, and **machine learning** to provide structured insights into news data.

The system extracts key entities and their relationships using **Named Entity Recognition (NER)** and **Relation Extraction** techniques, forming a **Knowledge Graph** that visually and semantically represents the connections among people, organizations, locations, and events. This helps uncover hidden relationships and patterns within the news corpus. **Community detection** and **centrality analysis** are then applied to identify influential nodes and clusters within the network, revealing how information or misinformation spreads through interconnected entities.

A **Question Answering (QA) module** allows users to interact with the Knowledge Graph, enabling intuitive information retrieval through natural language queries. To enhance the graph's accuracy and richness, **Cross-Domain Linking** integrates verified external knowledge bases, ensuring reliable contextual information.

The system also includes a **Fake News Classification** component that predicts the authenticity of news articles, distinguishing between real and fake content using linguistic and relational patterns. The entire framework is deployed using **Streamlit**, offering an interactive and user-friendly interface that visualizes the graph, displays analytics, and provides authenticity predictions.

Overall, this integrated approach not only detects misinformation but also promotes transparency and explainability in analysis, offering a powerful and scalable tool to combat fake news in the digital era.

CHAPTER 1 INTRODUCTION

In today's interconnected world, social media and online news platforms have become the primary channels for information dissemination. However, the rapid spread of unverified and misleading information has made it difficult to distinguish factual news from fabricated content. The consequences of fake news extend beyond individual misunderstanding — they influence public opinion, policy decisions, and societal stability.

The Knowledge Graph-based Fake News Detection and Analysis System aims to address this issue by leveraging the power of natural language processing and graph analytics. Knowledge graphs provide a structured representation of entities (such as people, organizations, or locations) and the relationships between them. By visualizing and analyzing these relationships, deeper insights can be derived regarding the nature and authenticity of news content.

This system not only detects misinformation but also provides analytical insights through graph-based visualization, enabling researchers, journalists, and general users to explore the hidden connections within data. In addition, a Question Answering (QA) system built on top of the knowledge graph allows for intuitive querying and exploration of the extracted information.

The implementation integrates multiple components — NER, Relation Extraction, Graph Construction, Community Detection, Centrality Analysis, Cross-Domain Linking, and Fake News Prediction — unified under an interactive Streamlit-based web application. Together, they form a robust framework capable of understanding, detecting, and presenting news data in a meaningful way.

1.2 SYSTEM SPECIFICATION

The **Knowledge Graph-Based Fake News Detection and Analysis System** integrates natural language processing (NLP), graph analytics, and machine learning techniques to detect and analyze fake news with high precision. To ensure efficient execution, the system requires a robust hardware configuration capable of handling large text datasets and computationally intensive graph-based operations. On the software side, a combination of powerful Python libraries and frameworks is used to perform data preprocessing, named entity recognition (NER), relation extraction, knowledge graph construction, and fake news classification. The overall architecture is designed to be modular, scalable, and optimized for deployment using Streamlit, ensuring a smooth and interactive user experience.

This section outlines the **hardware** and **software** specifications required to build and deploy the proposed system effectively.

1.2.1 HARDWARE SPECIFICATIONS

The hardware configuration determines the computational efficiency and scalability of the fake news detection and analysis system. Since the system involves extensive text processing, NLP model inference, and graph-based computations (such as centrality and community detection), adequate processing power and memory are essential. The hardware components should support high-speed computation and efficient data storage to ensure real-time analysis and smooth execution.

Recommended Configuration:

- **Processor:** Intel Core i5 or higher / AMD Ryzen 5 or higher
- **RAM:** Minimum 8 GB (16 GB recommended for large datasets)
- **Storage:** At least 100 GB free space for datasets, models, and temporary files
- **GPU:** NVIDIA GTX 1050 or higher (optional, but accelerates model training and NER tasks)
- **Display:** Standard HD display
- **Input Devices:** Keyboard, Mouse
- **Network:** Stable internet connection (for accessing APIs, model updates, and Streamlit deployment)

This configuration ensures that the system can efficiently handle complex NLP pipelines, large-scale text data, and graph visualization components without significant latency.

1.2.2 SOFTWARE SPECIFICATIONS

The software environment forms the backbone of the proposed system, providing tools and frameworks necessary for data preprocessing, NLP-based entity and relation extraction, knowledge graph construction, and fake news detection model deployment. A Python-based environment has been chosen due to its versatility, strong ecosystem for NLP and ML, and compatibility with visualization and web deployment frameworks.

Software Stack and Tools:

- **Operating System:** Windows 10 / 11 / Linux
- **Programming Language:** Python 3.8 or higher
- **Development Environment:** Jupyter Notebook / VS Code / Google Colab

Key Libraries and Frameworks:

- **Pandas:** For efficient data manipulation and preprocessing of textual datasets.
- **NumPy:** For numerical computations and handling large data arrays.

- **NLTK & spaCy:** For natural language processing tasks including tokenization, part-of-speech tagging, and named entity recognition.
- **Transformers (Hugging Face):** For advanced NLP models used in fake news classification and contextual analysis.
- **NetworkX:** For constructing, analyzing, and visualizing knowledge graphs, community structures, and centrality measures.
- **Matplotlib & Plotly:** For graph and data visualization.
- **Scikit-learn:** For implementing machine learning algorithms and fake news classification.
- **Streamlit:** For developing an interactive and user-friendly web-based deployment interface.

Together, these software tools and libraries enable seamless integration between machine learning, NLP, and graph-based components, ensuring the system is efficient, scalable, and easy to deploy.

CHAPTER II SYSTEM STUDY

2.1 Traditional System Problems

Traditional fake news detection systems rely heavily on manual verification, keyword-based filtering, or statistical models that consider only textual features. These methods face several limitations:

- **Lack of Context Understanding:** They analyze sentences in isolation, ignoring entity-level connections.
- **Scalability Issues:** Manual or semi-automated methods are slow and cannot handle large volumes of online data.
- **Limited Explainability:** Users are often not provided with the reasoning behind a classification decision.
- **Domain Restriction:** Models trained on specific datasets often fail to generalize to new topics or contexts.

These shortcomings highlight the need for a structured, context-aware, and interpretable system capable of understanding relationships between entities — which motivates the use of knowledge graphs.

2.2 Proposed System

The proposed system overcomes the limitations of traditional methods by integrating machine learning, NLP, and graph analytics into a unified framework. It focuses on:

- **Entity Extraction and Relationship Mapping:** Identifying key entities from news text and their interconnections.
- **Knowledge Graph Construction:** Structuring these entities and relations in a graph for semantic analysis.
- **Community and Centrality Analysis:** Detecting sub-groups and influential nodes to identify content patterns.
- **Fake News Detection Model:** Classifying news articles as fake or true using NLP-based models.
- **Interactive Deployment:** Providing a user-friendly Streamlit interface for real-time exploration and prediction.

This approach provides not just a prediction but an explainable representation of how news entities are interconnected, enabling trust and transparency in results.

2.2.1 Named Entity Recognition (NER) and Relation Extraction

Named Entity Recognition (NER) is a subtask of NLP that identifies and categorizes entities such as persons, organizations, locations, and dates from unstructured text. It transforms free-form text into structured data by tagging words with their semantic categories.

Relation Extraction follows NER and determines how these entities are related within a sentence or across sentences — for instance, identifying "X is the CEO of Y" or "A took place in B." These relationships are crucial for constructing meaningful connections in the knowledge graph.
Together, NER and Relation Extraction enable automatic understanding of semantic relationships between entities, forming the foundation for the knowledge graph.

2.2.2 Knowledge Graph

A **Knowledge Graph (KG)** represents entities as nodes and relationships as edges, capturing both syntactic and semantic connections between them.
Knowledge graphs help in visualizing information networks, identifying hidden patterns, and enabling knowledge-based reasoning. They are especially useful for news analysis, where events and actors are interconnected.

The constructed KG in this system supports query-based exploration, relationship inference, and graph-based analytics, making it an essential component for both explainability and discovery.

2.2.3 Community Detection

Community Detection is a graph analytical method used to identify clusters of closely connected nodes within a larger network. In the context of news data, communities may represent groups of related topics, entities involved in the same events, or networks of similar stories.
Detecting such clusters helps in understanding how information spreads and which entities are frequently mentioned together, potentially revealing sources or networks of misinformation.

2.2.4 Centrality Measures

Centrality Measures determine the importance or influence of nodes within a network. Various metrics, such as *degree centrality*, *betweenness centrality*, and *eigenvector centrality*, provide different perspectives on node significance.
In a news knowledge graph, high-centrality nodes may represent key persons, organizations, or events that are central to multiple stories. This helps identify major influencers or topics in the dataset.

2.2.5 Question Answering (QA) as an Application

The **Question Answering System** allows users to interact with the knowledge graph through natural language queries. Instead of searching manually, users can ask questions like “Who is associated with organization X?” or “Which locations are linked to event Y?”
By leveraging entity relationships in the KG, the QA module retrieves precise, context-aware answers, improving accessibility and understanding of the data.

2.2.6 Cross-Domain Linking

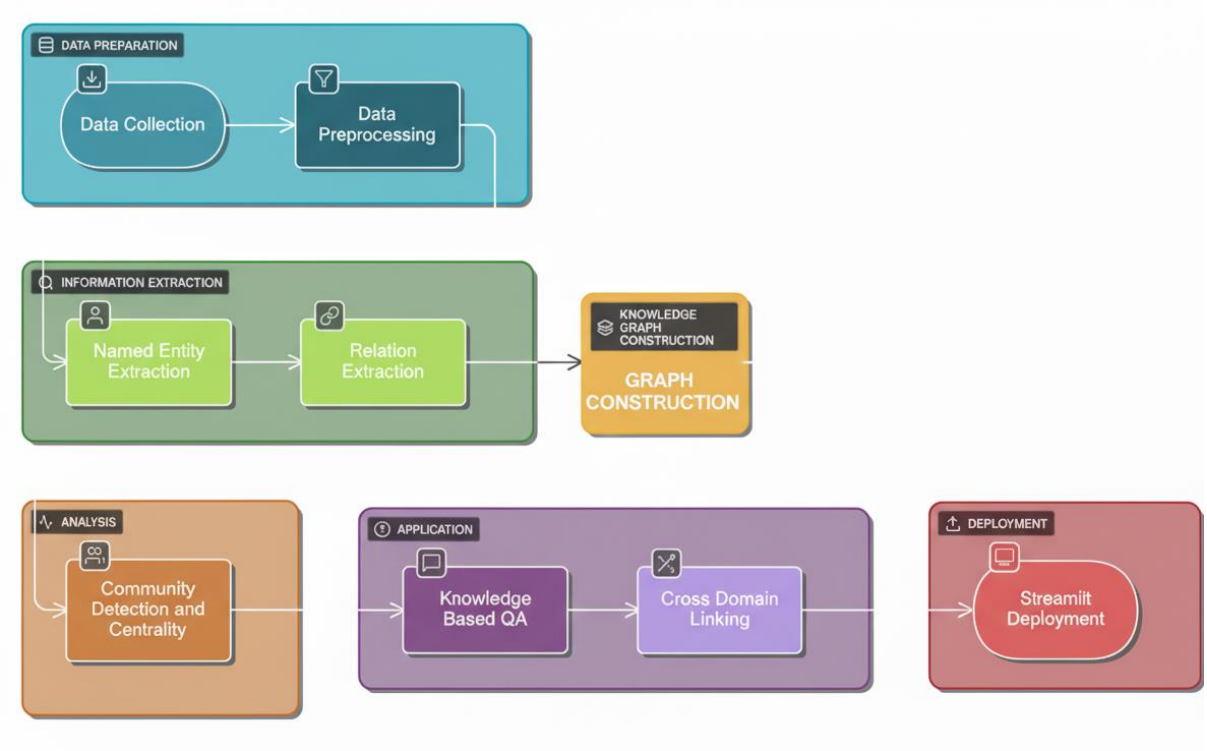
Cross-Domain Linking enhances the knowledge graph by connecting its entities with external data sources such as Wikipedia, DBpedia, or other domain-specific knowledge bases. This increases the coverage, accuracy, and depth of the graph, allowing the system to connect concepts across multiple contexts and domains.
For fake news analysis, cross-domain linking helps validate claims by comparing entities and facts with trusted knowledge bases.

2.2.7 Deployment Architecture

The system is deployed using **Streamlit**, a Python-based framework for building interactive web applications.
Streamlit enables seamless integration of NLP models, graph visualization components, and QA systems into an intuitive interface.
Users can upload news data, visualize entity relationships, perform QA, and check fake/true news predictions — all within a single web dashboard.

CHAPTER III SYSTEM DESIGN AND ARCHITECTURE

The workflow diagram for the development of the project is given in the following figure:



3.1 Dataset Collection

The foundation of any data-driven system lies in the quality and relevance of its dataset. For this project, which aims to perform fake news detection and knowledge graph-based information extraction, the dataset plays a crucial role in training, analysis, and validation.

3.1.1 Source of Data

The dataset used in this project was obtained from publicly available fake news detection repositories that contain labeled news articles. These datasets typically include a collection of headlines and article bodies that are annotated as either **"true"** or **"fake"**, allowing for supervised machine learning.
The sources were chosen based on their credibility and diversity, covering multiple topics such as politics, entertainment, and global affairs to ensure a balanced representation of fake and legitimate content.

3.1.2 Dataset Description

Each record in the dataset generally contains:

- **Title or Headline:** The short text summarizing the main idea of the news.
- **Text/Body:** The full content or description of the news article.
- **Label:** The truthfulness category assigned to each news item — either True or Fake.

Some datasets also include metadata such as the publisher, date, and author, though these were not the main focus of the initial phase. The dataset primarily emphasized text content for linguistic and semantic analysis.

3.1.3 Data Volume and Structure

The dataset consists of several thousand news samples, balanced across both fake and true labels. The size was sufficient to perform effective training and testing of machine learning models while ensuring computational efficiency.
The dataset was stored in **CSV (Comma Separated Values)** format to facilitate easy loading and manipulation in Python. Pandas was used extensively for reading, exploring, and preprocessing the data.

3.2 Data Preprocessing

Data preprocessing is one of the most crucial stages in the system pipeline, directly influencing the performance, accuracy, and interpretability of the final model. Since the dataset contains raw textual data gathered from different news sources, it often includes inconsistencies, noise, and irrelevant information. Therefore, several preprocessing operations were performed systematically to convert unstructured text into a machine-readable and semantically consistent format.

3.2.1 Objective of Preprocessing

The primary objectives of preprocessing were:

- To clean and normalize textual data for better model understanding.
- To remove unnecessary symbols, punctuations, and inconsistencies.
- To standardize case formatting and tokenize words for natural language processing.
- To prepare the dataset for both **machine learning-based fake news prediction** and **knowledge graph entity extraction**.

This ensured that the data pipeline feeding into later modules (NER, relation extraction, and model training) remained robust, scalable, and accurate.

3.2.2 Handling Missing and Duplicate Data

The first step involved ensuring dataset integrity. The raw dataset was loaded using the **Pandas** library, after which exploratory analysis was performed to identify missing or duplicate entries.

- **Missing Values**
  Rows containing missing text or labels were removed, as incomplete data could lead to misclassification or hinder entity extraction.

- **Duplicate Entries:**
  Duplicate news articles were identified using Pandas' `duplicated()` function and removed to prevent data leakage and overfitting during model training.

By removing redundant and incomplete records, the dataset achieved a more balanced and representative form for both learning and analysis.

3.2.3 Text Cleaning and Normalization

News articles often contain a wide range of non-informative characters such as punctuation, URLs, HTML tags, and special symbols. These were systematically cleaned using **Regular Expressions (regex)** and **string manipulation** functions.

Steps included:

- **Lowercasing:**
  All text was converted to lowercase to maintain uniformity, as models treat uppercase and lowercase words as distinct tokens otherwise.
- **Removing URLs and HTML Tags:**
  URLs, hashtags, and HTML markup were removed using regex patterns to eliminate unnecessary web-related noise.
- **Punctuation and Special Character Removal:**
  Non-alphabetic characters such as punctuation marks, digits, and emojis were stripped, retaining only meaningful textual content.
- **Whitespace Normalization:**
  Multiple spaces were replaced with a single space for consistency.

This process ensured that the text was clear, linguistically valid, and free from distractions that could mislead tokenization or feature extraction.

3.2.4 Tokenization and Stopword Removal

After text normalization, each sentence was broken down into smaller units called tokens using NLP libraries such as **NLTK** or **SpaCy**.
Tokenization splits sentences into individual words, which allows deeper linguistic analysis and statistical feature extraction.

Next, **stopwords** — common but semantically weak words (like the, is, of, and) — were removed. This reduces dimensionality and helps the model focus on more informative terms that contribute to distinguishing fake from true news.

- By eliminating stopwords, the model learns from the most relevant parts of speech, improving both the interpretability and predictive performance of later components.

3.2.5 Lemmatization and Stemming

- The next step involved reducing words to their base or root forms using **lemmatization** and **stemming**.
- **Lemmatization** uses linguistic knowledge to reduce words to their meaningful root forms (e.g., "running" → "run", "better" → "good"), ensuring grammatical validity.
- **Stemming**, on the other hand, performs a simpler truncation-based reduction, primarily focusing on reducing word variants.
- In this project, **lemmatization** was preferred since it preserves linguistic meaning — important for the contextual understanding required in NER and relation extraction.

3.2.6 Label Encoding and Data Balancing

- After cleaning the text, the target labels (Fake and True) were encoded into numeric form to make them compatible with machine learning algorithms. Using **LabelEncoder** from `sklearn.preprocessing`, the text labels were converted into integers — typically `0` for Fake and `1` for True.
- Class imbalance was checked using value counts. If one class significantly outweighed the other, resampling techniques such as **SMOTE (Synthetic Minority Oversampling Technique)** or **random undersampling** could be applied to ensure balanced training.
- Balanced data improves fairness and reduces bias in classification results.

3.2.7 Feature Extraction (TF-IDF / Embeddings)

- Once textual data was preprocessed, it was transformed into numerical features using **TF-IDF (Term Frequency–Inverse Document Frequency)** vectorization.
  TF-IDF assigns higher weights to words that are more discriminative, reducing the impact of overly frequent but uninformative terms.
- Alternatively, advanced embeddings like **Word2Vec**, **GloVe**, or **BERT** can be used for deeper semantic representation, though TF-IDF was optimal for the project's scope and interpretability.
- This step prepared the text for both classification and relation extraction models.

3.2.8 Data Splitting

- The processed data was divided into **training and testing sets** using an 80–20 split, ensuring that the model could be evaluated on unseen data.

3.3 Named Entity Recognition (NER)

Named Entity Recognition (NER) is a key component of the system, responsible for identifying and categorizing important entities within news articles. This process allows the system to detect **people, organizations, locations, dates, and other relevant entities**, which helps in differentiating fake news from real news based on entity patterns.

- **Implementation Approach**:
  The system utilized the **spaCy library** with the pre-trained en_core_web_sm language model to perform entity extraction. Each article's text was analyzed to identify entities and their corresponding categories.
- **Techniques Applied**:
  - Text normalization including lowercasing and removal of special characters.
  - Tokenization for sentence and word-level analysis.
  - Entity extraction and categorization based on pre-trained NLP models.
- **Outcome**:
  The NER process produced structured information from unstructured news text, enabling the system to generate features such as **entity counts per category**, which were later used to enhance classification and knowledge graph construction.

3.4 Relation Extraction and Exporting to Database

Following entity recognition, the system implemented **relation extraction** to identify **subject-verb-object (SVO)** relationships in the news text. These relationships represent interactions between entities, forming the backbone of the knowledge graph.

- **Implementation Approach**:
  Relation extraction was performed using **spaCy's dependency parsing** capabilities. Each sentence was analyzed to detect verbs and their associated subjects and objects, creating structured relational triples (subject, relation, object).
- **Database Integration**:
  Extracted triples were systematically stored in a **JSON-based database**, which acted as a persistent storage for knowledge graph construction and later retrieval for visualization or question answering.
- **Techniques Applied**:
  - SVO triple extraction for semantic understanding.
  - Noun-chunk analysis for additional relational context.
  - Confidence scoring assigned to each triple to indicate extraction reliability.
- **Outcome**:
  The structured relational data allowed the system to **link entities logically**, forming the foundation for knowledge graph construction and enabling semantic queries.

3.5 Knowledge Graph Construction

The system leveraged the extracted entities and relations to build a **knowledge graph**, representing the interconnected information within the news articles.

- **Implementation:**
  A **graph-based model** was constructed using **NetworkX**, where nodes represented entities, and edges represented relations. Edge labels captured the type of relationship between entities.
- **Techniques Applied**:
  - Graph representation for entity relationships.
  - Node and edge attribute assignment for metadata such as confidence scores and source articles.
- **Outcome**:
  The knowledge graph enabled a **visual representation of entity relationships**, facilitating analysis of news content, pattern detection, and downstream tasks like community detection.

3.6 Community Detection

Community detection was applied to the knowledge graph to identify clusters of closely related entities, revealing patterns that might indicate coordinated information or frequent associations in fake versus real news.

- **Implementation Approach**
  The **Louvain algorithm** was used for community detection, optimizing modularity to identify densely connected groups of entities.
- **Techniques Applied**:
  - Partitioning the graph into communities.
  - Assigning community IDs to each entity for analysis and visualization.
- **Outcome**:
  The resulting communities highlighted groups of entities frequently mentioned together, aiding in **structural analysis of news narratives** and detection of unusual patterns.

3.7 Centrality Measures

To determine the **importance of entities** within the knowledge graph, several centrality measures were computed.

- **Techniques Applied**:
    - **Degree Centrality**: Identified entities with the most direct connections.
    - **Betweenness Centrality**: Highlighted entities that act as bridges between different communities.
    - **Closeness Centrality**: Measured how close an entity is to all others, indicating potential influence.
- **Outcome**:
  Centrality analysis helped prioritize key entities in news articles and identify those that may play a central role in fake news propagation.

3.8 Question Answering (QA)

The system incorporated a **question answering module** to allow semantic queries over individual articles. Users could ask questions related to the text, and the system would provide context-aware answers.

- **Implementation Approach**:
    - Utilized a **transformer-based NLP model** for extractive QA.
    - Processed each article individually, using the article as context for the question.

3.9 Cross-Domain Linking

Cross-domain linking was implemented to identify semantically similar articles across different domains of fake news. This helped detect **repeated narratives or coordinated misinformation**.

- **Implementation Approach**:
    - Article embeddings were generated using **Sentence Transformers**.
    - Cosine similarity was computed between articles across domains.
    - Highly similar articles were linked to form cross-domain connections.

- **Outcome**:
  The system could trace **narrative propagation** across sources and identify articles sharing similar content, aiding deeper analysis of misinformation trends.

3.10 Deployment

The complete system was deployed as a **web-based application** using **Streamlit**, providing an interactive interface for news analysis.

- **Implementation Approach**:
  - Backend integration with the pre-trained **Random Forest model** and TF-IDF vectorizer for classification.
  - Integration of NER, relation extraction, knowledge graph, community detection, QA, and cross-domain linking.
  - JSON files served as persistent storage for knowledge triples and community Q&A data.
- **Outcome**:
  The deployed system provided a **complete end-to-end solution** for fake news detection, semantic analysis, and interactive exploration of news data.

CHAPTER IV: CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

In an era dominated by rapid information exchange, distinguishing between factual and fabricated content has become an increasingly complex challenge. The Knowledge Graph-Based Fake News Detection and Analysis System addresses this issue by combining Natural Language Processing (NLP), graph analytics, and machine learning techniques to analyze news articles more intelligently.

Through Named Entity Recognition (NER) and Relation Extraction, the system identifies key entities and their interconnections, transforming unstructured textual data into a structured Knowledge Graph. This graph-based representation enables deeper insight into news relationships, allowing for community detection, centrality analysis, and identification of influential entities.

In addition, the integration of a Fake News Prediction Model enhances the system's utility by classifying articles as "fake" or "real" using linguistic and contextual features. The deployment of the entire framework using Streamlit provides an intuitive and interactive platform for users to visualize results, query the graph, and understand misinformation patterns in real time.

Overall, the proposed system successfully demonstrates how semantic understanding, graph-based reasoning, and AI-driven classification can be integrated to combat misinformation effectively. It not only serves as a detection mechanism but also as an analytical tool for studying the dynamics of information dissemination.

6.2 FUTURE SCOPE

While the current system effectively detects and analyzes fake news using graph-based and NLP-driven techniques, there remains substantial room for future enhancement and research.

Some promising directions for future development include:

1. **Integration of Real-Time Data Streams:**
   Incorporating APIs from live news feeds or social media platforms (such as Twitter or Reddit) can enable the system to detect misinformation as it spreads in real time.
2. **Multilingual Fake News Detection:**
   Expanding the system's language capabilities beyond English by leveraging multilingual transformer models (like mBERT or XLM-R) to handle regional and vernacular content.

3. **Enhanced Knowledge Graph Scalability:**
   Adopting graph databases such as Neo4j or Amazon Neptune for improved scalability and faster querying as the graph grows in size and complexity.

4. **Sentiment and Emotion Analysis:**
   Integrating sentiment analysis can help correlate emotional tone with misinformation spread patterns, offering psychological insights into fake news propagation.

5. **Explainable AI (XAI) Integration:**
   Implementing interpretable models to provide clear reasoning behind the fake news classification decisions, enhancing transparency and user trust.

6. **Cross-Domain Linking Expansion:**
   Extending the system's cross-domain linking capability to connect data from multiple sources such as academic papers, verified fact-checking databases, and open data repositories.

7. **Deployment as a Cloud-Based Service:**
   Deploying the system on cloud platforms (AWS, Azure, or GCP) for public or institutional use, providing scalable access to researchers, journalists, and policymakers.

By implementing these enhancements, the **Knowledge Graph-Based Fake News Detection and Analysis System** can evolve into a comprehensive, scalable, and globally accessible solution to combat misinformation and promote reliable information dissemination in the digital age.